Lab 3: part 2

Course goals

- To implement and use data structures and algorithms in application programs.
- To analyze and evaluate various data structures and algorithms in solving computational problems with respect to efficiency and appropriateness.

Getting started

To get started with the exercise, perform the steps indicated below.

- Create a folder for this lab part named e.g. `Lab3-part2`.
- Download the zipped folder with the files for this exercise from the course website and unzip it into the lab folder.
- Navigate to the folder `detectionSystem` and execute the instructions in the file `README.md` (can be opened with Notepad). Your IDE should then show two projects, `LinesDiscovery` and `Rendering`.
- Select `LinesDiscovery` as startup project. Compile, link, and execute the program (the `main` is in `find-patterns.cpp`). When requested for a points file, enter e.g. `points200.txt`. As the system for detecting lines in a set of points is not implemented yet, the program produces no visible output. This implementation is assigned as part of an exercise.
- Select `Rendering` as startup project. Compile, link, and execute the program (the `main` is in `lab3-part2.cpp`). When requested for a points file, enter e.g. `points200.txt`. All points in the input file are then plotted. Note that this program only provides plotting functionality, for both input points and line segments[1]. You don't need to modify this program — it's ready to be used.
- Read the remaining lab description.
- Review lecture 8.

If you have any specific question about the exercises, then send us an e-mail. Be short and concrete, otherwise you won't get a quick answer. You can write your e-mail in Swedish. Add the course code to the e-mail's subject, i.e. "`TND004: ...`".

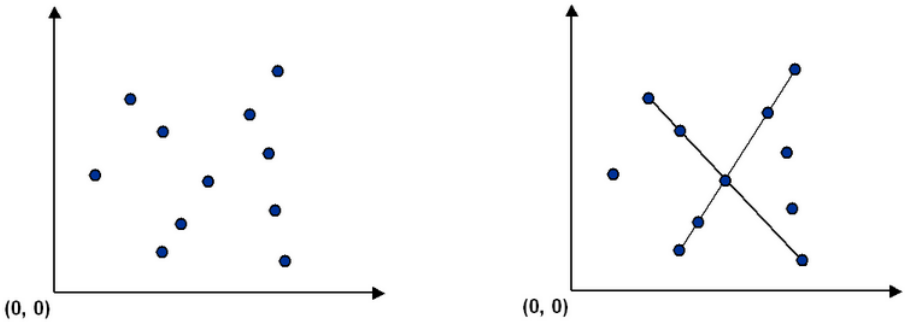
Exercise 1: To find patterns in large datasets of points

In many real-world applications, data is captured as collections of points, whether from sensors, images, or spatial measurements. A fundamental challenge is to infer the underlying structures that generated those points. **Identifying collinear points** is a key step in this process, as they often reveal the presence of linear features or alignments in the data. For example, in geographic information systems (GIS), groups of collinear points may represent roads, pipelines, rivers, or power lines — features critical for mapping and spatial analysis. In interactive environments like computer games, detecting collinear arrangements can enable strategic behaviors, such as targeting multiple enemies along a straight path.

[1] At the start of this exercise, there are no line segments yet discovered from the points files. Thus, no line segments are plotted, yet.

This exercise focuses on precisely that problem:

1. Given a dataset of $n > 0$ distinct $2D$ points, find every line segment that connects four or more of them, as illustrated below.



The brute-force approach would be to check all combinations of 4 points, which requires $O(n^4)$ time, and is clearly inefficient for large n.

```
for (i1 = 0; i1 < size(points) - 3; ++i1) {
    for (i2 = i1 + 1; i2 < size(points) - 2; ++i2) {
        for (i3 = i2 + 1; i3 < size(points) - 1; ++i3) {
            for (i4 = i3 + 1; i4 < size(points); ++i4) {
                if (onSameLine(points[i1], points[i2], points[i3], points[i4])) {
                    // …
                }
            }
        }
    }
}
```

It is possible to solve the problem much faster than the brute force solution. An algorithm is presented in the next section.

A sorting-based algorithm

For each point p, compute the slopes it forms with all other points and sort those points by slope relative to p. Any group of three or more adjacent points with equal slopes in this order is collinear with p, meaning that together they form a line segment containing at least four collinear points. Sorting ensures that points sharing the same slope with respect to p appear consecutively.

The idea above is used in the following algorithm:

```
1.  Input: List of N distinct points in 2D (x, y)
2.  Output: All maximal line segments that connect 4 or more collinear points
3.  for each point p in points:
4.      - Create a copy C of the remaining points (excluding p)
5.      - Sort C by the slope the points in it make with p
6.      - Scan through the C to find contiguous groups of ≥ 3 points with the same slope
7.              If a group is found:
8.                  - Add p to the group
9.                  - Sort the group of points (including p)
```

Implement the algorithm above. Add your code to the file `find-patterns.cpp` within the `linesDiscovery` project. The first and last points in each sorted group of points (line 9) determine a line segment to be rendered, and these two points should

be written to an output file (see section "Output data files"). Ensure that each line segment appears only once in the output file.

Input data files

Several input files with 2D-points are provided in the folder `detectionSystem/data`. (e.g. `points1.txt`, `points5.txt`, `largeMystery.txt`). These text files contain points coordinates which are integers in the interval $[0, 32767]$. Each input file starts with the number of points ($n > 0$) followed by the points coordinates, one point per line. You can assume there are no duplicate points in the input files.

Output data files

Given an input points file, your program should produce an output text file with the line segments to be rendered.

- Name the file based on the input file. For instance, if the input file is named `points6.txt` then the line segments file must be named `segments-points6.txt`. Place the file in the `detectionSystem/data/output` folder[2] so the rendering program can locate it.
- Each line in this file should contain the coordinates of two points, four integers in total, representing the endpoints of a line segment (see example below).

```
10000 0 0 10000
3000 4000 20000 21000
```

Rendering system

To render the points and line segments, select `Rendering` as startup project and execute the program.

In the folder `detectionSystem/data/output/line-plots`, you can also find `png` files with the plot of the line segments discovered from the corresponding points files, with exception for the input file `largeMystery.txt`.

Hints

- Standard library functions in C++ should be utilized for sorting operations.
- Three points $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$, and $p_3 = (x_3, y_3)$ are in the same line (i.e. are collinear), if $((y_2 - y_1) \times (x_3 - x_1)) = ((y_3 - y_1) \times (x_2 - x_1))$. Note that these expressions do not involve floating-point computations and should be used whenever possible.
- The sorting (based on slope) in step 5 of the algorithm involves floating-point comparisons[3]. You don't need to take any special action for this.

Exercise 2: Avoiding per-group sorting via global point ordering

Reconsider again the algorithm described in *"A sorting-based algorithm"*. The goal is to avoid repeatedly sorting each detected collinear group in line 9, which is currently done to determine the segment's endpoints for rendering.

---

[2] There is also a subfolder named `lines-discovered` containing the lines found, including all data points on each line (not only the endpoints), which can be useful for debugging purpose.
[3] The slope should be a `double`.

Sort the input points in advance (i.e., before the loop in line 3), for example by coordinate y and then by x. What additional modifications to your implementation would make it possible to identify the endpoints of each collinear group without having to sort each group in line 9?

Discuss your ideas with the lab assistant during the *handledning* session, update your implementation accordingly, and ensure that each line segment is written only once in the output file.

Exercise 3: Time and space complexity analysis

Provide a written analysis of your program's time and space complexity after completing Exercise 2. In particular, show that your program runs in $O(n^2 \log n)$ time.

Presenting part2 and deadlines

The exercises in this lab part are compulsory and you should demonstrate your solutions during the lab session *Lab3 RE*. Read the instructions given in the labs webpage and consult the course schedule.

Necessary requirements for approving your lab are given below.

- The code must be readable, well-indented, and use good programming practices.
- Present your code for Exercises 1 and 2, and justify the modifications introduced in Exercise 2.
- Present and justify your time and space complexity analysis, as required in Exercise 3.
- If you consulted an AI assistant during this part, show examples of the prompts you used together with the corresponding responses you received.
- Answer the question: what pattern is hidden in the points file `largeMystery.txt`?

If your solutions for the exercises in lab3/part2 have not been approved in the scheduled lab session *Lab3 RE* then it is considered a late lab. Late labs can be presented provided there is time in a another RE lab session. All groups have the possibility to present one late lab on the extra RE lab session scheduled in the end of the course.